Lctr 7: Bitwise Operations and Direct GPIO Register Access

Jianhua Liu

Spring 2026

==Topics==

Core concepts and applications:

- Shift and bitwise operations in C.
- GPIO control via direct register access in C.
  - Setting GPIO mode.
  - Setting GPIO output type.
  - Setting GPIO input pull-up, pull-down, or no-pull.
  - Reading GPIO IDR and writing GPIO ODR.

Demo project:

- Controlling a GPIO pin using direct GPIO register access.
- Using SysTick timer (a special timer that will be discussed later) to do two works *in parallel*.

7.1 Introduction

In this lecture, we first review C's shift and bitwise operations and then use them to directly access the GPIO registers to operate the GPIO pins. These operations will be illustrated in a demo project. These operations include the following:

- Modifying partial contents of a register while keeping others untouched via bitwise operations.
- Seeing the standard three-step operation: load, modify, and store.
- Seeing the flexibility of hardware control using registers.

7.2 Demo project

The main functionalities of this project, `de2f_direct_grio_red_access`, are as follows for G431N32:

- Setting PB8 as a digit output, for driving LED4, using CubeMX. This is the same as `dc2s_gpio_key_n_led__uart`.
- Initializing PA10 as a digit output, for driving LED1 on the game controller board, by directly writing the following registers (use default for other registers):
  - GPIOA's MODER (mode register).
  - GPIOA's OTYPER (output type register).
- Initializing PA0 as a digit input, for reading Fnc6, by directly writing the following registers (use default for other registers):
  - GPIOA's MODER (mode register).
  - GPIOA's PUPDR (pull-up / pull-down register).
- Reading PA0's value by directly reading GPIOA's IDR.
- Toggling the LED driving digital output pins to blink the LEDs by directly accessing the ODR (output data register) of GPIOB.

7.3 Shift and bitwise operations in C

Shift and bitwise operations are widely used in C, especially embedded programming in C. With these operations, we can specifically operate on selected bits in a variable or register.

7.3.1 Shift operations

We have seen that we can shift a number to the left by one bit to double its value. Now, let's discuss C's shift operations systematically. For convenience of the presentation, we use hexadecimal expressions here.

Assume `uhA = 0xABCD` is a `uint16_t` variable and `ihB = 0xABCD` a `int16_t` one. Now, if we perform 4-bit right shifting to them:

```
uhA = uhA >> 4;
ihB = ihB >> 4;
```

The results will be `uhA = 0x0ABC` and `ihB = 0xFABC`, respectively. The reason is that

- `uhA` is an unsigned number, and we always **shift in 0s from the left side** of the number when doing right shifting.
- `ihB` is an signed number, and we always **repeat the sign bit** when doing right shifting. The value of `ihB` is `0xABCD`, meaning the sign bit is 1, so that we need to repeat this bit 4 times. (If the value of `ihB` was `0x7BCD`, then the shifted result will be `0x07BC` as the sign bit is 0 for this case.)

When we shift the numbers to the left, the **always shifting 0s from the right side**. For example, if we perform 4-bit left shift to them:

```
uhA = uhA << 4;
ihB = ihB << 4;
```

The results will both be `0xBCD0`. Note that we can completely change sign of the signed number during left shifting, which can cause disastrous results.

7.3.2 Basic single-bit logic operations

7.3.2.1 Truth table of four basic single-bit logic operations

Assume both `a` and `b` are single-bit logic variables, which can be **Boolean variables** or **single bits in normal variables** at the **same bit position**. We have the following **truth table** for FOUR single-bit logic operations: AND, OR, XOR (EOR, Exclusive OR), and NOT:

a	b	a AND b	a OR b	a XOR b	NOT a
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

The above single-bit logic operations can be extended to bitwise operations on other data types that have more than 1 bits.

7.3.2.2 Toggling a single bit

From the above truth table, we can reach the following conclusions:

- a XOR 0 <=> a.
- a XOR 1 <=> NOT a.

The second operation can be used to toggle a bit. The above XOR operations will be used later to toggle some bits and keep some other bits unchanged, using a **bit mask**, as discussed below.

7.3.3 Bitwise operators

The C bitwise operators are listed below together with their precedence and associativity.

Precedence	Operator	Description	Associativity
2	~	Bitwise NOT	Right to left
8	&	Bitwise AND	Left to right
9	^	Bitwise XOR	Left to right
10	|	Bitwise OR	Left to right
14	&=, ^=, |=	Assignment by bitwise operation	Right to left

7.3.4 Bit mask

A bit mask is such an unsigned variable that **has 1s on those bits that we intend to change** and 0s on those bits that we intend to keep unchanged. Here is an example. Suppose we have a `uint_8` variable `A` for which we want to change the least significant THREE bits while keeping the other bits unchanged. Then, we can configure a `uint8_t` mask `M` shown below (assuming we can assign *verbose* binary numbers to variables):

```
uint8_t M = 0b0000_0111;
```

Many times, if we consider 32-bit operations, we just write `M = 7U`. **Here `U` stands for unsigned long**, or `uint32_t`. If we intend to work on bit 3 to bit 5 (the LSB is bit 0), we can use `M = 7U << 3` since the three 1's are now shifted to the positions of bit 3 to bit 5 to have the least significant 8 bits being `M = 0b0011_1000`.

7.4 Basic applications of bitwise applications

7.4.1 Using bitwise AND to check a bit from a variable or register

In real applications, we often need to check a single bit of a variable or register. By using the AND operation with a mask, we can easily read a single bit from a variable or register.

A	b_7	b_6	b_5	b_4	b_3	b_2	b_1	b_0
mask = 1 << N	0	0	1	0	0	0	0	0
A & mask	0	0	b_5	0	0	0	0	0

Figure 1: Reading a bit using AND with a mask.

```
bool mp_read_a_bit(uint32_t A, int N) {
    uint32_t mask = 1U << N;
    A &= mask;
    bool bit_N = A > 0;
    return bit_N;
}
```

7.4.2 Using bitwise OR to set a bit of a variable or register to 1

By using the **bitwise-OR** operation with a mask, we can easily **set** a single bit (force it to 1) of a variable or register, as shown in Figure 2.

A	b_7	b_6	b_5	b_4	b_3	b_2	b_1	b_0
mask = 1 << N	0	0	1	0	0	0	0	0
A |= mask	b_7	b_6	1	b_4	b_3	b_2	b_1	b_0

Figure 2: Setting a bit using OR with a mask.

Here is a function in C for **setting bit `N` of a variable to 1 without changing the other bits**.

```
uint32_t mp_set_a_bit(uint32_t A, int N) {
    uint32_t mask = 1U << N;
    A |= mask;
    return A;
}
```

7.4.3 Using bitwise AND with the inverted mask to clear a bit of a variable or register to 0

By using the **bitwise-AND** operation with an **inverted mask**, we can easily **clear** a single bit (force it to 0) of a variable or register without affecting other bits, as shown in Figure 3.

A	b_7	b_6	b_5	b_4	b_3	b_2	b_1	b_0
~mask	1	1	0	1	1	1	1	1
A &= ~mask	b_7	b_6	0	b_4	b_3	b_2	b_1	b_0

Figure 3: Clearing a bit using AND with an inverted mask.

Here is a function in C for **clearing bit `N` of a variable to 0 without affecting the other bits**.

```
uint32_t mp_clear_a_bit(uint32_t A, int N) {
    uint32_t mask = 1U << N;
    A &= ~mask;
    return A;
}
```

7.4.4 Assigning several bits to a variable or register

Assigning a number of bits has to be done in two steps:

- Step 1. Clear these bits using a mask.
- Step 2. Assign these bits using the bitwise-OR operation with the **masked and shifted** value intended to assign.

A	b_7	b_6	b_5	b_4	b_3	b_2	b_1	b_0
~mask	1	1	0	0	0	1	1	1
A &= ~mask	b_7	b_6	0	0	0	b_2	b_1	b_0
v = mask & (v << s)	0	0	1	0	1	0	0	0
A |= v	b_7	b_6	1	0	1	b_2	b_1	b_0

Figure 4: Assigning several bits.

```
uint32_t mp_assign_bits(uint32_t A, int s, int w, uint32_t v)
    {
     uint32_t mask = (1U << w) - 1;
     mask <<= s;
```

```
    A &= ~mask;
    A |= mask & (v << s);
    return A;
}
```

7.4.5 Using bitwise XOR to toggle several bits of a variable or register

By using the XOR (EOR) operation with a multi-bit mask, we can easily toggle a number of bits of a variable or register, as shown in Figure 5.

A	b_7	b_6	$\mathbf{b_5}$	b_4	$\mathbf{b_3}$	b_2	b_1	b_0
mask = 0x28	0	0	**1**	0	**1**	0	0	0
A ^= mask	b_7	b_6	$\bar{\mathbf{b}}_5$	b_4	$\bar{\mathbf{b}}_3$	b_2	b_1	b_0

Figure 5: Toggling several bits using EOR with a multi-bit mask. $\bar{b}_n$ is the inverse of b_n.

Here is a function in C for **toggling a number of bits indicated by a mask without changing the other bits**.

```
uint32_t mp_toggle_bits(uint32_t A, uint32_t mask) {
    A ^= mask;
    return A;
}
```

Note that for the above case in Figure 5, we need to have the following value for `mask` for toggling pins 3 and 5:

```
mask = (1U << 3) | (1U << 5);
```

7.5 Direct access of GPIO registers

Here, **we will use the demo project to discuss how to access the GPIO registers directly** to perform the needed operations.

7.5.1 MODER—the GPIO mode register

The detailed bit map and bit control of MODER are shown in Figure 6.

We can see that each GPIO pin requires two bits in MODER to express its four modes:

- `0b00`: GPIO input mode.
- `0b01`: GPIO output mode.
- `0b10`: Alternate function mode.
- `0b11`: Analog mode (default).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODE15[1:0]		MODE14[1:0]		MODE13[1:0]		MODE12[1:0]		MODE11[1:0]		MODE10[1:0]		MODE9[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODE7[1:0]		MODE6[1:0]		MODE5[1:0]		MODE4[1:0]		MODE3[1:0]		MODE2[1:0]		MODE1[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **MODE[15:0][1:0]:** Port x configuration I/O pin y (y = 15 to 0)
These bits are written by software to configure the I/O mode.
00: Input mode
01: General purpose output mode
10: Alternate function mode
11: Analog mode (reset state)

Figure 6: Bit map of MODER and control details.

Assume we want to set the mode of PA10 (GPIOA Pin 10). We need to change the values of MODE10, which can be done by performing one of the following operations, according to the desired mode:

- `0b00`: GPIO input mode

```
GPIOA->MODER &= ~(3U << 10 * 2);
```

- `0b01`: GPIO output mode

```
GPIOA->MODER &= ~(3U << 10 * 2);
GPIOA->MODER |= (1U << 10 * 2);  // 1U = 0b01
```

- `0b10`: Alternate function mode

```
GPIOA->MODER &= ~(3U << 10 * 2);
GPIOA->MODER |= (2U << 10 * 2);  // 2U = 0b10
```

- `0b11`: Analog mode (default)

```
GPIOA->MODER &= ~(3U << 10 * 2); // optional
GPIOA->MODER |= (3U << 10 * 2);  // 3U = 0b11
```

7.5.2 OTYPER—the output type register

The detailed bit map and bit control of OTYPER are shown in Figure 7. We can see that each GPIO pin requires one bit in OTYPER to express its two types:

- 0: GPIO output push-pull mode (default)
- 1: GPIO output open-drain mode

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 7: Bit map of OTYPER and control details.

To set the output mode of PA10 to open-drain, we need to set the OTYPER bit, OT10, to 1, which can be done by using

```
GPIOA->OTYPER |= mask_Pin10; // mask_Pin10 = (1U << 10);
```

Yet, we need to use the **push-pull** mode, which is default, and hence we cannot run the above line of code, which is used for illustration purposes.

Since PA0 works as input, the setup of OTYPER is not applicable to it.

7.5.3 OSPEEDR—the GPIO port output speed register

There are 4 different speeds to select using this register, which needs 2 bits for each pin. As such, the control is very much like for MODER. If we want to reduce the power, we need to use lower speeds.

The default value is good PA10 for the application in the demo project, and we will ignore the control of this register. As for PA0, it works as input, the setup of OSPEEDR is not applicable to it.

7.5.4 PUPDR—the pull-up/pull-down register

The detailed bit map and bit control of PUPDR are shown in Figure 8. We can see that each GPIO pin requires two bits in PUPDR to express its three modes:

- `0b00`: No pull-up or pull-down
- `0b01`: Pull-up mode
- `0b10`: Pull-down mode
- `0b11`: Reserved

Reset value: 0x6400 0000 (for port A)
Reset value: 0x0000 0100 (for port B)
Reset value: 0x0000 0000 (for other ports)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PUPD15[1:0]		PUPD14[1:0]		PUPD13[1:0]		PUPD12[1:0]		PUPD11[1:0]		PUPD10[1:0]		PUPD9[1:0]		PUPD8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PUPD7[1:0]		PUPD6[1:0]		PUPD5[1:0]		PUPD4[1:0]		PUPD3[1:0]		PUPD2[1:0]		PUPD1[1:0]		PUPD0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 8: Bit map of PUPDR and control details.

As we need to configure PA0 as pull-down, which reads 0 when Fnc6 is not pressed, we can use the following code

```
// PUPDR: PA0 = pull-down, value = 0b10
GPIOA->PUPDR &= ~(3U << 0 * 2);
GPIOA->PUPDR |= (2U << 0 * 2);
```

7.5.5 IDR—the input data register

The detailed bit map of IDR is shown in Figure 9.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ID15	ID14	ID13	ID12	ID11	ID10	ID9	ID8	ID7	ID6	ID5	ID4	ID3	ID2	ID1	ID0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Figure 9: Bit map of IDR.

To read a single pin, say PA0, we can use the following code:

```
// mask_Pin0 = (1U << 0);
bool PA0_state = ((GPIOA->IDR & mask_Pin0) == mask_Pin0);
```

or the following

```
bool PA0_state = (GPIOA->IDR & mask_Pin0) > 0
```

7.5.6 ODR—the output data register

The detailed bit map of ODR is shown in Figure 10.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OD15	OD14	OD13	OD12	OD11	OD10	OD9	OD8	OD7	OD6	OD5	OD4	OD3	OD2	OD1	OD0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 10: Bit map of ODR.

To write 1 and 0 to PA10 and PA8, respectively, we can use the following code:

```
GPIOA->ODR |= mask_Pin10;    // mask_Pin10 = (1U << 10);
GPIOA->ODR &= ~mask_Pin8;    // mask_Pin8 = (1U << 8);
```

To toggle these two pins, we can use:

```
GPIOB->ODR ^= mask_Pin10 | mask_Pin8;
```

7.5.7 BSRR—bit set/reset register

The detailed bit map of BSRR is shown in Figure 11. Looking carefully at the contents of this register, which is writable only, we can see that it is divided into two sections:

- The BS (bit set) section used for setting an ODR bit to 1.
- The BR (bit reset or clear) section for clearing a bit to 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Figure 11: Bit map of BSRR.

The reason of using this register is to save the time for operation. We have discussed that the following two statements used to set PA10 and clear PA8 each needs the load, modify, and store steps:

```
GPIOA->ODR |= mask_Pin10;    // mask_Pin10 = (1U << 10);
GPIOA->ODR &= ~mask_Pin8;    // mask_Pin8 = (1U << 8);
```

Using the BSRR, we can use the following statements to perform the same work:

```
GPIOA->BSRR = mask_Pin10;   // mask_Pin5 = (1U << 10);
GPIOA->BSRR = mask_Pin8R;   // mask_Pin8R = (1U << (8+16));
```

Each of the above two statements for BSRR uses two steps:

- Load a mask to a general-purpose register.
- Store the value from the general-purpose register to the BSRR.

Note that **to reset a bit, we need to use a mask that is shifted to the left by 16 more bits as the bits for bit reset are on the upper 16 bits**.

7.5.8 BRR—bit reset register

With this register, the above bit reset operation, reproduced here

```
GPIOA->BSRR = mask_Pin8R;  // mask_Pin8R = (1U << (8+16));
```

can be rewritten as

```
GPIOA->BRR = mask_Pin8;    // mask_Pin8 = (1U << 8);
```

which is consistent with the other operations.

7.5.9 Other registers

There are two more registers for most of the STM32 MCUs.

We will not discuss these registers as they are not used as often as the other registers in the class.

7.6 Additional reading

7.6.1 Logical operators

We have discussed bitwise operators. C has related operators, called **logical operators**, `&&`, `||`, and `!`, which **work on the entire number**, not bitwise. This means that these logical operators assume the operands are BOOLEAN; as long as the operand is not zero, it is considered 1.

7.6.2 Using a timer to control the toggling of the output pins

7.6.3 GPIO pin initialization

The original code of `HAL_GPIO_Init(...)` uses a very efficient data structure to hold the information collected from CubeMX to set up the GPIO pins. Yet, it is not easily readable.

We have provided multi-step incremental revisions to make it very readable. See the Companion for details. You are encouraged to read these code revisions to see how to refactor the code to a more readable form and understand how a data structure is used in the initialization function to bridge the setups in CubeMX to the operations of the GPIO registers.